UNIT-V: Monitoring, Visualization and Logging Middleware

Monitoring Middleware: Observability Concepts: Metrics, logs, and traces – understanding their differences and importance, Prometheus Architecture: Prometheus server, Time-Series Data Model, Scraping and Pull-Based Metrics, Exporters, Introduction to PromQL, Alerting Concepts for critical events in distributed applications.

Visualization Middleware: Grafana Architecture, Data Source Integration with Prometheus and Elasticsearch, Dashboard Creation, Use of Panels and Variables, Alert Configuration and Notifications, Role of Grafana: Operational monitoring, system analysis, and performance insights.

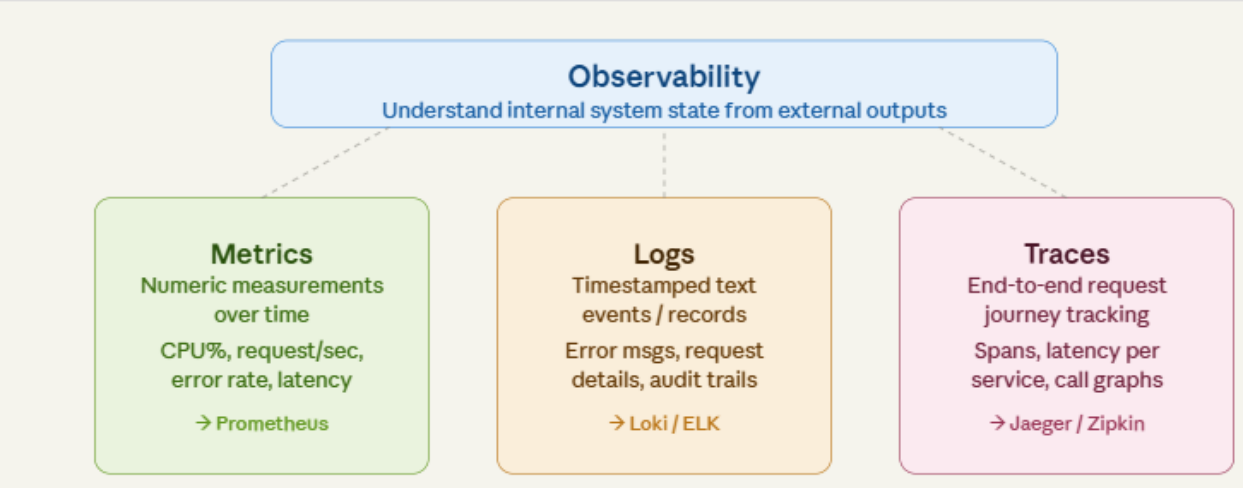
Logging Middleware: Need for Centralized Logging in Distributed Systems, Loki Architecture, Log Streams and Labels, Introduction to LogQL, LogQL Queries, Integration of Loki with Grafana, Comparison of Loki-based logging with Traditional ELK Stack.

Observability Concepts

Modern software is deployed as distributed systems — dozens or hundreds of microservices running across multiple servers, containers, and cloud regions. When something goes wrong, you need to quickly understand *what happened, where it happened, and why*. This ability to understand the internal state of a system by examining its external outputs is called **Observability**.

Observability rests on three pillars — often called the **Three Pillars of Observability**:

Figure 1.1 — The Three Pillars of Observability



Metrics

Metrics are numerical measurements collected over time, representing a specific aspect of a system's behavior or performance. They are typically aggregated data points, such as CPU utilization, memory consumption, request rates, error rates, and latency. Metrics are highly efficient for storage and querying. Metrics answer questions like "How many requests per second is my server handling?" or "What is the average response time over the last 5 minutes?". They answer the fundamental question: "What happened?"

Types of Metrics:

- **Counter:** A value that only increases. Example: total HTTP requests served, total errors.
- **Gauge:** A value that can go up or down. Example: current memory usage, active connections.
- **Histogram:** Samples observations and counts them in configurable buckets. Example: request duration distribution.
- **Summary:** Similar to histogram, but calculates quantiles on the client side. Example: 95th percentile latency.

REAL-WORLD EXAMPLE

An e-commerce site uses metrics to track: number of orders placed per minute (counter), current items in the shopping cart service queue (gauge), and checkout response time distribution (histogram). When the histogram shows 99% of users wait more than 3 seconds, the operations team is alerted immediately.

Logs

Logs are **discrete, immutable, timestamped records of events** that happen within a system. They provide rich contextual information — who did what, when, and what error occurred. Unlike metrics (which aggregate), logs capture individual events. Logs are crucial for understanding "Why it happened?".

- **Structured Logs:** JSON-formatted logs with key-value pairs. Easy to parse and query.
- **Unstructured Logs:** Plain text lines. Human-readable but harder to process programmatically.
- **Semi-structured Logs:** Combination with patterns like Apache Combined Log Format.

Example — Structured Log (JSON)

{"timestamp":"2024-11-15T10:32:01Z","level":"ERROR","service":"payment-api","user_id":"u_9", "error": “Payment gateway timeout","duration_ms":5002,"trace_id":"abc"}

Traces

A trace represents the **complete journey of a single request** as it travels through a distributed system. A trace is made up of multiple **spans**, where each span represents a single unit of work (e.g., a database query, an API call to another service).

- **Trace ID:** Unique identifier shared across all spans of a single request.
- **Span:** A named, timed operation representing a piece of the trace.
- **Parent Span:** The caller service's span that initiated a child span.

REAL-WORLD EXAMPLE

A user clicks "Buy Now". The trace captures: Frontend → API Gateway (12ms) → Order Service (45ms) → Inventory Service (8ms) → Payment Service (320ms timeout!) → Response. The trace immediately pinpoints that the payment service caused the failure — something metrics alone couldn't reveal.

Differences at a Glance

Feature	Metrics	Logs	Traces
Purpose	What happened? (High-level overview)	Why it happened? (Detailed events)	Where it happened? (End-to-end request flow)
Data Type	Numerical, aggregated time-series	Textual, discrete events	Graph of spans, distributed context
Structure	Highly structured, dimensional	Unstructured to highly structured (JSON)	Structured, hierarchical (parent-child spans)
Tools	Prometheus, InfluxDB	Loki, ELK Stack	Jaeger, Zipkin, Grafana
Storage Cost	Low (aggregated)	High (volume-dependent)	Moderate (often sampled)
QuerySpeed	Fast (for aggregated data)	Slower (for detailed searches)	Fast (for specific request flows)
Use Cases	Monitoring, alerting, trend analysis	Debugging, auditing, forensic analysis	Distributed tracing, bottleneck identification

Prometheus Architecture

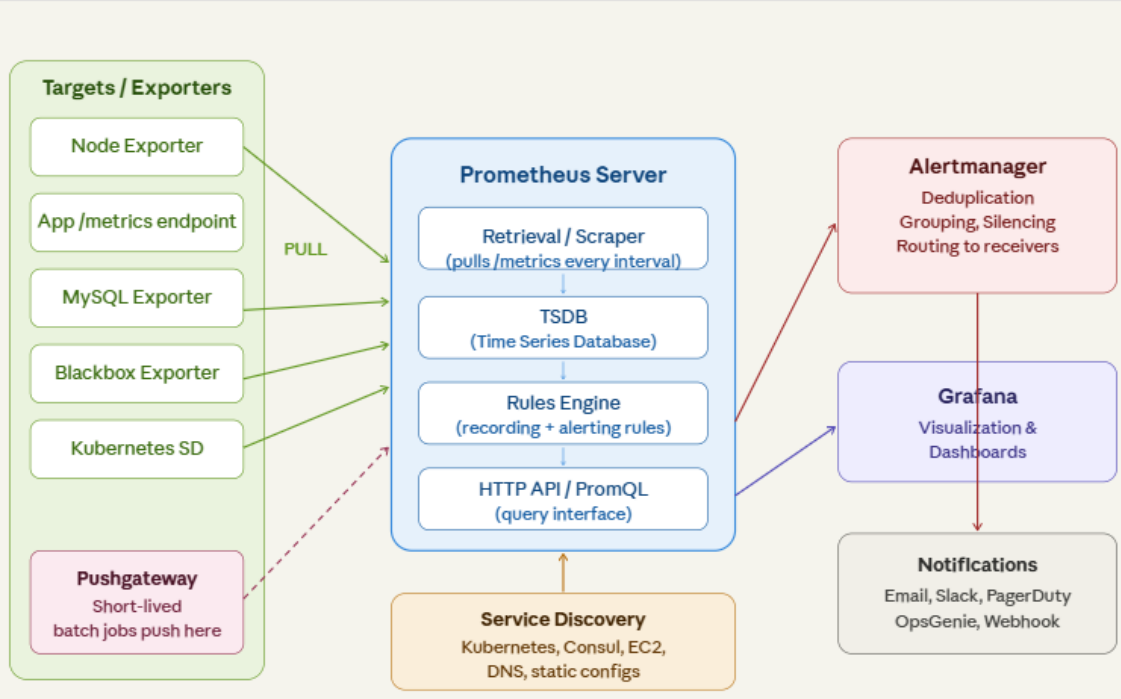
Prometheus is an open-source systems monitoring and alerting toolkit originally developed at SoundCloud. It has since become a standalone open-source project and a graduated project of the Cloud Native Computing Foundation (CNCF). Prometheus is designed for reliability and provides a robust solution for collecting and storing metrics as time-series data, making it a popular choice for monitoring modern, dynamic environments like microservices and containerized applications.

Core Architecture

The Prometheus ecosystem comprises several components, with the core being the Prometheus server. The architecture is primarily pull-based, meaning the Prometheus server actively scrapes metrics from

configured targets. The following diagram illustrates the key components and their interactions within the Prometheus ecosystem:

Figure 2.1 — Prometheus Architecture Overview



Key Components:

- **Prometheus Server**: The central component responsible for scraping, storing, and querying time-series data.
- **Exporters**: Agents that expose metrics from third-party systems or applications in a Prometheus-compatible format.
- **Pushgateway**: An optional intermediary for pushing metrics from short-lived jobs that cannot be scraped directly.
- **Alertmanager**: Handles alerts sent by the Prometheus server, deduplicating, grouping, and routing them to appropriate receivers.
- **Client Libraries**: Used to instrument application code directly to expose custom metrics.
- **Service Discovery**: Mechanisms (e.g., Kubernetes, Consul, file-based) that allow Prometheus to dynamically discover monitoring targets.
- **Visualization Tools**: Such as Grafana, used to visualize the collected data through dashboards.

Prometheus Server

The Prometheus server is the central component. It has several critical functions:

1 **Data Retrieval (Scraper)**: The server is configured to discover targets and periodically pull (scrape) metrics from them over HTTP. This pull-based model is a fundamental design choice of Prometheus, offering several advantages over push-based systems, such as easier target management and better control over the scraping process.

2 **Time-Series Database (TSDB)**: Prometheus stores all scraped samples locally in its highly efficient, custom-built time-series database. This TSDB is optimized for high ingestion rates and efficient querying of time-series data. It is designed to be autonomous, meaning each Prometheus server operates independently without relying on distributed storage, which enhances its reliability during outages.

3 **HTTP Server**: The Prometheus server exposes an HTTP API that allows users to query the stored data using PromQL (Prometheus Query Language) and provides a built-in expression browser for ad-hoc querying and visualization.

4 **Rule Processing**: The server processes configured recording rules and alerting rules. Recording rules pre-compute frequently needed or expensive query results and store them as new time series. Alerting rules define conditions under which alerts should be fired and sent to the Alertmanager.

Time-Series Data Model

Every metric in Prometheus is stored as a **time series** — a stream of timestamped numerical values. A time series is uniquely identified by its **metric name** and a set of **labels** (key-value pairs). This dimensional data model is highly flexible, allowing for powerful filtering and aggregation of metrics.

Metric Name: Describes the general feature being measured (e.g., http_requests_total). Metric names should be descriptive and follow naming conventions (e.g., [a-z A-Z_:] [a-zA-Z0-9_:] *).

Labels: Allow for different instances of the same metric to be distinguished. For example, http_requests_total{method="POST", handler="/messages"} identifies a specific time series for HTTP POST requests to the /messages endpoint. Any change in a label's value, or the addition/removal of a label, creates a new, distinct time series

Data Model Syntax

Scraping and Pull-Based Metrics

Prometheus uses a **pull model** — it actively fetches metrics from targets rather than waiting for targets to push data. This is in contrast to tools like Graphite or InfluxDB which use push-based models.

How scraping works:

- Each target exposes a /metrics HTTP endpoint in a text-based format.
- Prometheus scrapes each target at a configurable interval (default: 15 seconds).
- Scraped data is parsed and written to the TSDB.

Why pull-based is better for monitoring

Pull-based scraping allows Prometheus to detect if a target is down (scrape fails), centralizes configuration (all targets defined in one place), and avoids network flooding from thousands of agents pushing simultaneously.

prometheus.yml — basic scrape config

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'node-exporter'
        static_configs:
        - targets: ['localhost:9100']
 - job_name: 'my-webapp'
        static_configs:
        - targets: ['app-server:8080']
```

Exporters

Exporters are programs that **expose metrics from third-party systems** in a Prometheus-compatible format. Since most existing software (databases, OS, hardware) doesn't natively expose a /metrics endpoint, exporters act as translators.

Role of Exporters:

Standardization: They convert system-specific metrics into the Prometheus text-based exposition format, typically served over HTTP on a /metrics endpoint.

Integration: They enable Prometheus to integrate with and monitor a vast ecosystem of software, hardware, and services that were not originally designed for Prometheus.

Exporter	What it monitors	Default Port
Node Exporter	Linux/Unix hardware, OS metrics (CPU, memory, disk, network)	9100
MySQL Exporter	MySQL/MariaDB database metrics	9104
PostgreSQL Exporter	PostgreSQL database metrics	9187
Blackbox Exporter	HTTP, HTTPS, DNS, TCP, ICMP endpoint probing	9115
Redis Exporter	Redis cache metrics	9121
JMX Exporter	Java Virtual Machine (JVM) metrics via JMX	5556
cAdvisor	Docker container resource usage	8080

PromQL and Alerting for Critical Events

In a distributed system, monitoring is not just about collecting data; it's about making that data actionable. PromQL (Prometheus Query Language) is the powerful engine that allows you to filter, aggregate, and transform raw metrics into meaningful insights. Combined with Alerting Concepts, it forms the backbone of incident response for critical events.

Introduction to PromQL

PromQL is a functional query language that treats all data as time series. Understanding the difference between data types is the first step toward effective querying.

Core Data Types

Type	Description	Example
Instant Vector	A set of time series containing a single sample for each, all sharing the same timestamp.	http_requests_total
Range Vector	A set of time series containing a range of data points over time.	http_requests_total[5m]
Scalar	A simple numeric floating-point value.	10

Essential Functions for Critical Events

For distributed applications, we rarely care about absolute counter values; we care about the rate of change.

- **rate()**: Calculates the per-second average rate of increase. Always use this for alerts as it smooths out spikes and handles counter resets.

  Query: rate(http_requests_total[5m])

- **histogram_quantile()**: Essential for measuring latency. It calculates percentiles (like P99) from histogram buckets.

Query: histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

Common PromQL Examples

PromQL Queries with Explanations

 # Select all time series for a metric

```
http_requests_total
```

 # Filter by label

```
http_requests_total{job="api-server", status="200"}
```

```
# Rate of requests per second over last 5 minutes
rate(http_requests_total[5m])
# Total error rate across all services
sum(rate(http_requests_total{status=~"5.."}[5m]))
# CPU utilization percentage per core
100 - (avg by(cpu) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
# 95th percentile request duration
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
# Memory usage in MB
node_memory_MemAvailable_bytes / 1024 / 1024
# Services with error rate > 1%
(rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])) > 0.01
```

Alerting Concepts for Distributed Applications

Alerting is the mechanism by which a monitoring system **notifies humans or automated systems when something requires attention**. In distributed systems, effective alerting is critical — there can be hundreds of services, any of which can fail at any time.

Prometheus Alerting Rules

Alerting in Prometheus is defined using **alerting rules** in YAML files. When a rule's PromQL expression evaluates to true for a defined duration, Prometheus fires the alert to the Alertmanager.

alerting_rules.yml — Example Alert Definitions

```
groups:
  - name: critical_alerts
        rules:
        - alert: HighErrorRate
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
        for: 2m
        labels:
        severity: critical
        annotations:
        summary: "High error rate on {{ $labels.job }}"
        description: "Error rate is {{ $value }} errors/sec on job {{ $labels.job }}"

        - alert: ServiceDown
        expr: up == 0
        for: 1m
        labels:
        severity: page
        annotations:
        summary: "Service {{ $labels.instance }} is down"
```

Alertmanager

The Alertmanager receives alerts from Prometheus, and handles **deduplication, grouping, silencing, inhibition, and routing** to the right notification channel.

- **Deduplication:** Multiple Prometheus servers firing the same alert won't generate multiple notifications.
- **Grouping:** Alerts from the same cluster can be grouped into a single notification.
- **Silencing:** Suppress alerts during planned maintenance windows.
- **Inhibition:** If a critical alert fires, suppress related minor alerts.
- **Routing:** Send different alerts to different receivers (email, Slack, PagerDuty) based on labels.

SLOs and SLAs — Why Alerting Matters

SLI (Service Level Indicator): A measurable metric — e.g., request success rate.

SLO (Service Level Objective): A target for your SLI — e.g., 99.9% success rate.

SLA (Service Level Agreement): A legal commitment to your SLO with customers.

Alerts should fire when you're at risk of breaching an SLO, not just when something is technically wrong.

Real-World Use Case — Netflix

Netflix runs thousands of microservices. They configure Prometheus alerts based on "error budget burn rate" — if the current error rate will exhaust their monthly SLO error budget within 1 hour, a critical alert pages the on-call engineer immediately.

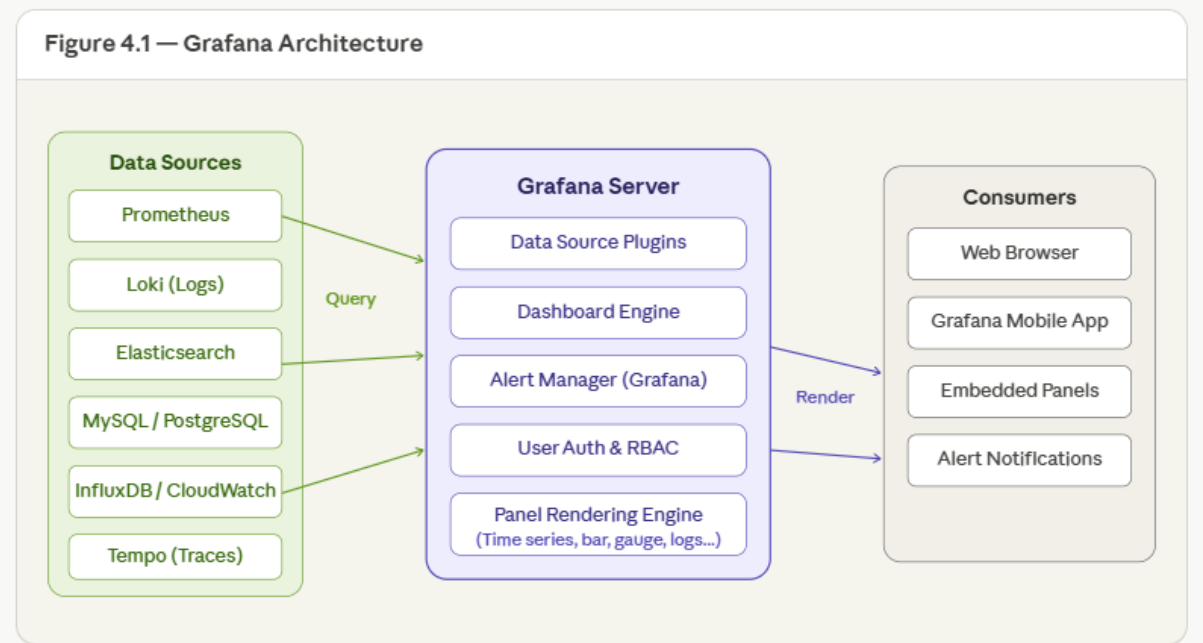
Visualization Middleware: Grafana

Grafana is an open-source analytics and interactive visualization platform. It connects to multiple data sources and allows you to build rich, real-time dashboards, set up alerts, and explore data. It is the most widely used visualization layer in modern observability stacks.

Grafana Architecture

The architecture is organized into three main sections:

- Data Sources — where the raw data lives
- Grafana Server — the brain that queries, processes, and renders
- Consumers — the interfaces where users interact with dashboards and alerts



Data Flow

The high-level data flow follows a simple three-step pattern:

Data Sources → Query → **Grafana Server** → Render → **Consumers**

When a dashboard loads or an alert rule evaluates, Grafana queries the appropriate data source, processes the results, and renders the output to wherever the user is consuming it.

1. Data Sources

Data sources are external systems that store and serve the raw data. Grafana never stores this data itself — it queries these systems on demand. Each source is specialized for a particular type of data.

Data Source	Type	What It Does
Prometheus	Metrics	Scrapes numeric metrics (CPU, memory, request rates) every few seconds from your apps and infrastructure.
Loki	Logs	Indexes log lines without parsing them upfront, making it fast and cheap. Uses LogQL for querying.
Elasticsearch	Search & Analytics	Full-text search and analytics engine. Useful for application logs and business events.
MySQL / PostgreSQL	Relational Data	Traditional relational databases. Grafana runs SQL queries directly against your tables.
InfluxDB / CloudWatch	Time-Series & Cloud	InfluxDB is purpose-built for time-series data. CloudWatch provides AWS metrics and logs.
Tempo	Distributed Traces	Stores traces — breadcrumb trails of every hop a request makes through your microservices.

2. Grafana Server

The Grafana Server is the core processing engine. It receives queries from dashboards and alerts, communicates with data sources via plugins, processes results, and renders panels. It contains five key internal components that work in a pipeline.

2.1 Data Source Plugins

Each data source has a dedicated plugin that acts as a translator. When a panel runs a query, the plugin converts it into the native language of the target source:

- PromQL for Prometheus
- LogQL for Loki
- SQL for MySQL / PostgreSQL
- TraceQL for Tempo

This plugin layer means Grafana can support virtually any data source without changing its core engine — you can even write custom plugins.

2.2 Dashboard Engine

The Dashboard Engine manages the layout, state, and behavior of dashboards. It is responsible for:

- Organizing panels into rows and grid layouts
- Resolving template variables (e.g. $environment, $host) before queries are sent
- Managing time ranges, refresh intervals, and zoom levels
- Linking panels together so clicking one filters the others

2.3 Alert Manager

The Alert Manager evaluates alert rules on a configurable schedule. It works by:

- Running the alert query against the data source
- Comparing the result against the defined threshold
- Changing the alert state (Normal → Pending → Firing → Resolved)
- Routing firing alerts to the correct contact points (Slack, email, PagerDuty, webhooks, etc.)

Alerts are defined per panel or in a dedicated alerting UI, and can be grouped and silenced as needed.

2.4 User Auth & RBAC

Grafana has a built-in authentication and authorization system. It controls who can see and do what:

- Supports local users, OAuth (Google, GitHub, Azure AD), LDAP, and SAML
- Role-Based Access Control (RBAC) allows fine-grained permissions per dashboard, folder, or data source
- Viewer: can see dashboards. Editor: can modify. Admin: full control
- Enterprise versions support team-level access and data source restrictions

2.5 Panel Rendering Engine

Once the data source returns results, the Panel Rendering Engine transforms raw data arrays into visual charts. It supports:

- Time series graphs

- Bar charts and histograms
- Gauge and stat panels
- Heatmaps and candlestick charts
- Log panels and table panels

Panels render in the browser using JavaScript/Canvas. For alert images or PDF exports, Grafana uses a headless browser renderer.

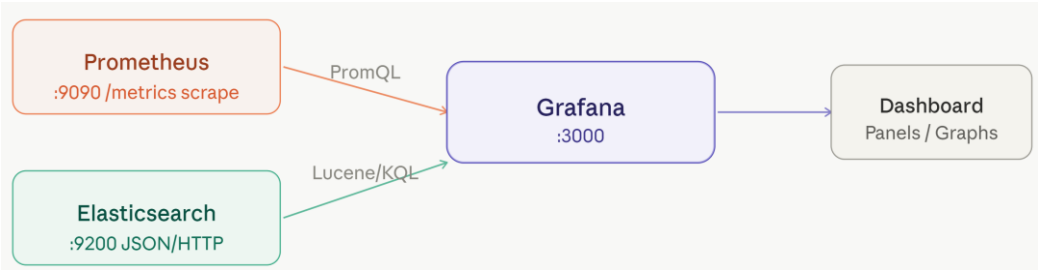
3. Consumers

Consumers are the end interfaces where the rendered output is delivered. Grafana supports multiple consumer types simultaneously.

Consumer	Description
Web Browser	The main Grafana UI. Dashboards update live via WebSocket — no page refresh needed. Panels are rendered client-side in the browser.
Grafana Mobile App	Official iOS/Android app for viewing dashboards on the go. Supports annotations and alert browsing.
Embedded Panels	Individual panels can be embedded into other websites or internal tools via an <iframe> with a public snapshot URL or authentication token.
Alert Notifications	When an alert fires, Grafana sends notifications through configured contact points: Slack, email, PagerDuty, OpsGenie, Microsoft Teams, or any custom webhook.

Data Source Integration with Prometheus and Elasticsearch

Grafana acts as a visualization layer — it connects to external data sources via plugins. Below are step-by-step guides for Prometheus and Elasticsearch.



Part A — Prometheus Integration

1 Open Data Sources in Grafana

Navigate to Home→Connections→Data Sources in the left sidebar, then click +Add new data source**.**

2 Select Prometheus plugin

In the search box type *Prometheus*. Select the built-in Prometheus plugin — it ships with Grafana by default (no install needed).

3 Configure the connection URL

Set the Prometheus server URL. Use the network-reachable host and port of your Prometheus instance.

http://localhost:9090

or inside Docker/K8s:

http://prometheus-service:9090

4 Set scrape interval & HTTP settings

Match the Scrape interval to your Prometheus scrape interval (default 15s). Enable Basic auth or TLS if your instance is secured.

Scrape interval: 15s

Query timeout: 60s

HTTP method: POST (recommended for large queries)

5 Save & Test

Click Save & Test. You should see a green banner: *"Successfully queried the Prometheus API."* If not, check that Grafana can reach the Prometheus host on port 9090.

Part B — Elasticsearch Integration

1 Add Elasticsearch data source

Go to **Connections → Data Sources → + Add new data source**, search for *Elasticsearch* and select it.

2 Enter the cluster URL

Provide the full HTTP/HTTPS URL of your Elasticsearch cluster. If secured, toggle **Basic authentication** and provide credentials.

URL: https://my-es-cluster:9200

Username: elastic

Password: ••••••••

3 Configure index pattern & version

Set the **Index name** (supports wildcards), the **Time field** used for time-based queries, and your Elasticsearch **version**.

Index name: logstash-*

Time field: @timestamp

Version: 8.x

4 Set min time interval

Set **Min time interval** to align with how frequently your logs are ingested (e.g., 10s for near-real-time log pipelines).

5 Save & Test

Click **Save & Test**. A successful connection shows *"Index OK. Time field name OK."*

Dashboard Creation

A Grafana dashboard is a collection of panels arranged on a grid. Here is the full workflow to create one from scratch.

1 Create a new dashboard

Click the + icon in the left sidebar → **New Dashboard**. Alternatively, go to **Dashboards** → **New** → **New Dashboard**.

2 Set dashboard title and folder

Click **Dashboard settings** (gear icon, top-right). Set a meaningful **Title** (e.g., *Infrastructure Overview*). Organize into a **Folder** for access control.

3 Add your first visualization

Click + **Add visualization**. Choose your data source (Prometheus or Elasticsearch). The panel editor opens with a query builder and visual preview.

4 Configure the time range

Use the time picker (top-right of dashboard) to set the range: *Last 1 hour*, *Last 24 hours*, or a custom absolute range. Set **Auto refresh** (e.g., every *30s*) for live monitoring.

5 Arrange panels on the grid

Drag panels to reposition. Grab the bottom-right corner to resize. Grafana uses a 24-column grid — plan your layout accordingly (full-width = 24 cols, half = 12, quarter = 6).

6 Save the dashboard

Press **Ctrl+S** (or click the Save icon). Add a commit message for version history. Grafana stores previous dashboard versions — you can roll back under **Dashboard settings** → **Versions**.

Dashboard Settings Reference

General: Title, description, tags, folder assignment

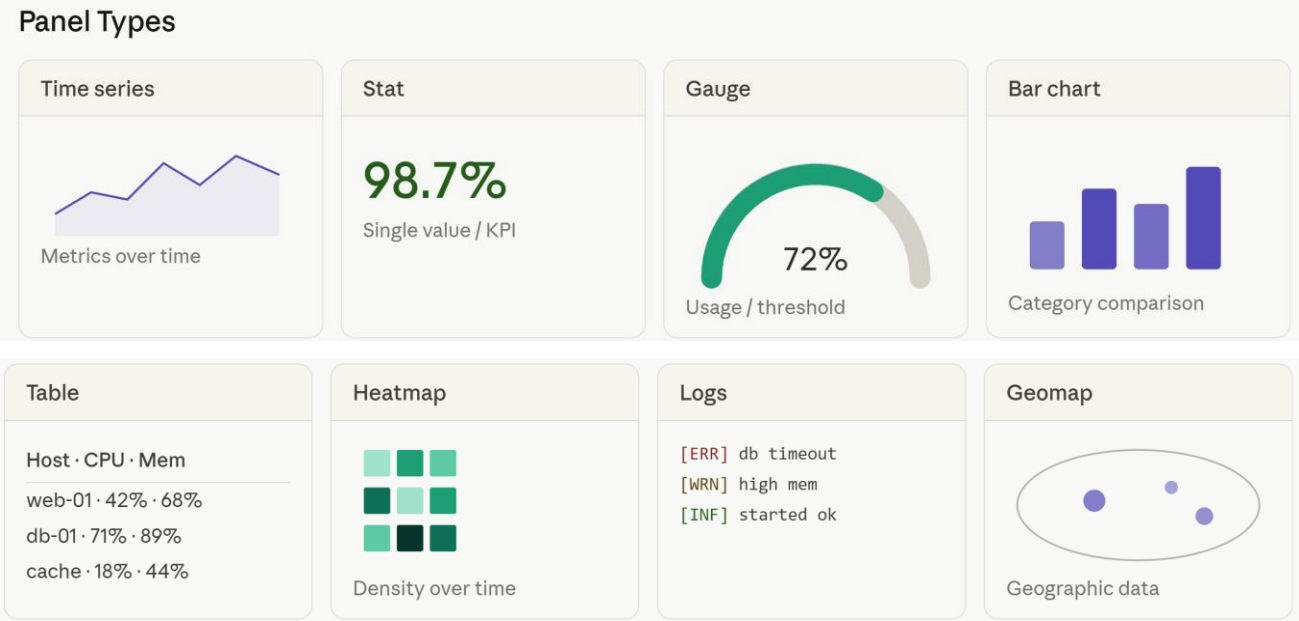
Time options: Default time range, refresh intervals, timezone

Variables: Template variables for dynamic filtering

Permissions: Viewer / Editor / Admin per team or user

Panels & Variables

Panels are the building blocks of every dashboard. Variables let you create dynamic, reusable dashboards.



Panel Editor — Step by Step

1 Open panel editor

Click **Add visualization** or hover a panel and click the **⋮ menu → Edit**. The panel editor splits into query builder (bottom) and preview (top).

2 Write your query

For Prometheus use PromQL. For Elasticsearch use the visual query builder or switch to raw Lucene/KQL.

```
# PromQL — CPU usage by instance
100 - (avg by(instance)(rate(
  node_cpu_seconds_total{mode="idle"}[5m]
)) * 100)

# Elasticsearch — error count
{ "query": { "match": { "level": "ERROR" } } }
```

3 Choose visualization type

In the right panel sidebar, click the visualization type dropdown (default: *Time series*). Switch to Stat, Gauge, Table, Bar chart, etc. based on your metric type.

4 Configure display options

Under **Panel options**, set Title, Description, and Transparent background. Under **Standard options**, configure Unit (e.g., *percent*, *bytes*), Min/Max, and Decimals.

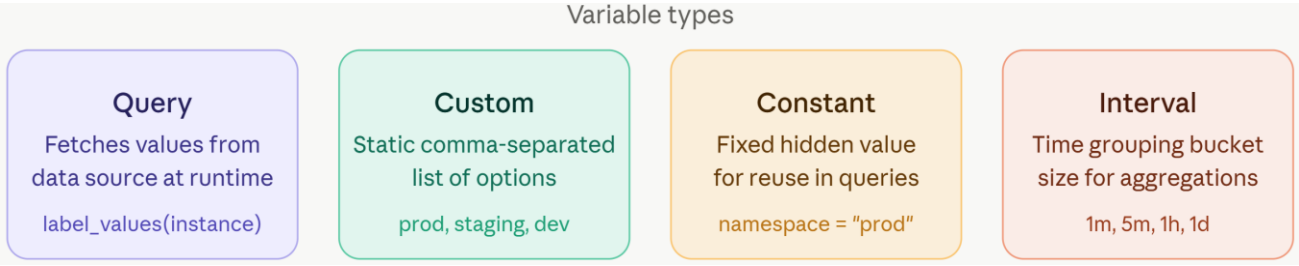
5 Add thresholds

In the **Thresholds** section, add color-coded lines. Example for CPU: Green below 70%, Yellow 70–90%, Red above 90%.

Green - 0→Yellow - 70→Red - 90

Template Variables

Variables make dashboards dynamic — users can switch between servers, namespaces, or environments using dropdowns.



1 Open Variables settings

Go to **Dashboard settings → Variables → + New variable**.

2 Create a Query variable (example: server list)

Set type to **Query**, choose your Prometheus data source, and use a label query to populate options dynamically.

Variable name: instance

Type: Query

Data source: Prometheus

Query: label_values(node_cpu_seconds_total, instance)

Refresh: On time range change

3 Reference the variable in panels

Use ${variable_name} syntax in PromQL or

Elasticsearch queries. Enable **Multi-value** and **Include All option** for group selections.

 # Use variable in PromQL

node_memory_MemAvailable_bytes{instance="$instance"}

 # Multi-select — regex join

node_load1{instance=~"$instance"}

4 Chain variables (dependent dropdowns)

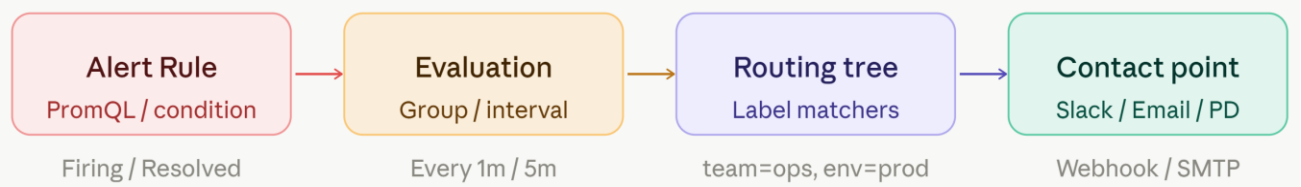
Reference one variable inside another's query to create parent → child filtering (e.g., Region → Cluster → Instance).

 # Cluster variable filtered by region

label_values(kube_pod_info{region="$region"}, cluster)

Alert Configuration & Notifications

Grafana Alerting (unified alerting) uses a **rule → evaluation group → contact point → notification policy** pipeline.



Step 1 — Create an Alert Rule

1 Navigate to Alerting → Alert rules

Click the **Bell icon** in the left nav → **Alert rules** → **+ New alert rule**.

2 Write the alert query (Grafana-managed)

Select your data source and write the expression. Use **Classic condition** or **Math / Reduce / Resample** expressions to define the threshold.

```
# Query A — PromQL
avg(rate(node_cpu_seconds_total{mode!="idle"}[5m])) * 100

# Condition: WHEN avg() OF A IS ABOVE 85
```

3 Set evaluation behavior

Assign to an **Evaluation group** (folder + interval). Set **Pending period** — how long the condition must be true before firing (avoids alert flapping).

```
Folder:          Infrastructure
Evaluation group: system-checks
Evaluate every:  1m
For (pending):   5m
```

4 Add labels and annotations

Labels are used by the routing tree to match contact points. **Annotations** add human-readable context to notifications.

```
Labels:
  severity = critical
  team     = ops
Annotations:
  summary     = "CPU above 85% on {{ $labels.instance }}"
  description = "Current value: {{ $values.A.Value | printf '%.1f' }}%"
```

Step 2 — Configure Contact Points

1 Create a contact point

Go to **Alerting → Contact points → + New contact point**. Grafana supports Slack, Email, PagerDuty, OpsGenie, Webhook, Teams, and more.

2 Configure Slack

Select **Slack**, paste the **Incoming Webhook URL**, set the channel, and customize the message template.

```
Webhook URL:  https://hooks.slack.com/services/…
Channel:      #ops-alerts
Title:        [{{ .Status | toUpper }}] {{ .CommonAnnotations.summary }}
Message:      {{ .CommonAnnotations.description }}
```

3 Configure Email

Requires SMTP configuration in grafana.ini. In the contact point, set recipient addresses and subject.

```
# grafana.ini SMTP section
[smtp]
enabled  = true
host     = smtp.gmail.com:587
user     = alerts@yourorg.com
password = ••••••••
from_address = grafana@yourorg.com
```

4 Test the contact point

Click **Test** to send a sample notification. Verify delivery before saving. Save the contact point.

Step 3 — Notification Policies (Routing)

1 Open Notification policies

Go to **Alerting → Notification policies**. The root policy is the default — all alerts not matched by nested policies go here.

2 Add nested policies with label matchers

Click **+ New nested policy**. Use label matchers to route specific alerts to specific contact points. Policies are evaluated top-down.

```
Matcher:         severity = critical
Contact point:   PagerDuty-oncall
Group by:        alertname, instance
Group wait:      30s
Group interval:  5m
Repeat interval: 4h
```

3 Configure silences (maintenance windows)

Go to **Alerting → Silences → + New silence**. Set a time range and label matchers. Matching alerts are suppressed during the silence window — useful for planned maintenance.

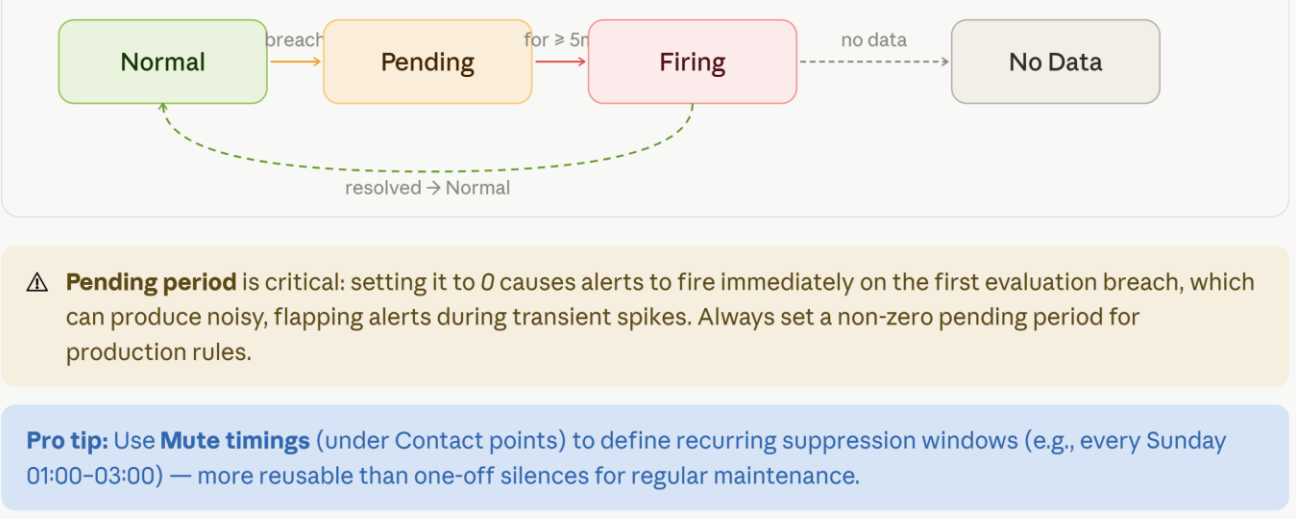
Start: 2026-04-20 02:00

End: 2026-04-20 04:00

Matchers: env = production

Comment: Planned database migration

Alert State Machine



⚠ **Pending period** is critical: setting it to *0* causes alerts to fire immediately on the first evaluation breach, which can produce noisy, flapping alerts during transient spikes. Always set a non-zero pending period for production rules.

Pro tip: Use **Mute timings** (under Contact points) to define recurring suppression windows (e.g., every Sunday 01:00–03:00) — more reusable than one-off silences for regular maintenance.

Role of Grafana: Operational monitoring, system analysis, and performance insights.

Grafana has emerged as the industry-standard "single pane of glass" for visualizing and analyzing data from disparate sources. Its role extends beyond simple charting, serving as a critical platform for operational monitoring, system analysis, and performance insights. By adhering to a "Big Tent" philosophy, Grafana allows organizations to unify their observability stack without being locked into a single vendor or data store.

Role in Operational Monitoring

Operational monitoring is the practice of maintaining real-time visibility into the health and availability of systems. Grafana facilitates this through several core capabilities:

Real-Time Visibility and Dashboards

Grafana's primary role is to transform raw, time-series data into actionable dashboards. It supports over 150 data sources, including Prometheus, InfluxDB, AWS CloudWatch, and Elasticsearch. This allows operators to monitor everything from infrastructure (CPU, memory, disk I/O) to application-level metrics (request rates, error codes) in a unified view.

Centralized Alerting

Grafana Alerting enables teams to define alert rules that span multiple data sources. This is a significant shift from traditional siloed monitoring where each tool has its own alerting logic.

- Flexible Routing: Alerts can be routed to different notification channels (Slack, PagerDuty, Email) based on severity or labels.
- Multi-Dimensional Alerting: Users can alert on specific instances or clusters using labels, reducing the need for hundreds of individual rules.

Infrastructure Health at a Glance

Using pre-configured dashboards (often provided via "Grafana Cloud" or community templates), teams can instantly gain visibility into standard stacks like Kubernetes, Linux servers, or databases.

Feature	Description	Impact on Operations
Multi-Source Support	Connects to SQL, NoSQL, and Time-Series DBs simultaneously.	Eliminates data silos and reduces context switching.
Live Streaming	Real-time data updates via WebSockets.	Critical for high-stakes monitoring during deployments.
Unified Alerting	One engine for all data sources.	Simplifies the management of incident response workflows.

Role in System Analysis

System analysis involves diving deep into historical data to understand behavior, identify trends, and perform root cause analysis (RCA). Grafana provides the tools to correlate disparate data types: metrics, logs, and traces.

The Three Pillars of Observability

Grafana excels at correlating the three pillars of observability, allowing an analyst to follow a problem from its symptom to its source:

1. **Metrics**: Identify that there is a problem (e.g., a spike in error rates in Prometheus).
2. **Logs**: Understand why it is happening (e.g., viewing specific error messages in Grafana Loki).
3. **Traces**: See where it is happening (e.g., tracing the request path in Grafana Tempo).

Data Transformations and Expressions

Grafana allows for sophisticated data manipulation at the visualization layer without requiring changes to the underlying database.

- **Transformations**: Users can join data from two different sources (e.g., joining a MySQL table with Prometheus metrics), rename fields, or apply mathematical operations.
- **Expressions**: This feature allows users to perform math across multiple queries, such as calculating the ratio of errors to total requests across two different systems.

Historical Context and Trend Analysis

By providing easy time-range selection and comparison tools, Grafana helps analysts distinguish between "normal" seasonal patterns and genuine system anomalies.

Role in Performance Insights

Performance insights focus on optimizing systems, identifying bottlenecks, and ensuring that Service Level Objectives (SLOs) are met.

Advanced Visualizations for Performance

Standard line graphs are often insufficient for performance tuning. Grafana provides specialized panels:

- **Heatmaps**: Essential for visualizing latency distributions. A line graph showing "average latency" can hide outliers; a heatmap reveals the "long tail" of slow requests.
- **Node Graphs:** Used to visualize the topology of microservices, showing the flow of traffic and where latency is accumulating between services.
- **Flame Graphs:** Integrated via Grafana Phlare (Profiling), these help developers identify "hot paths" in their code that consume excessive CPU or memory.

Capacity Planning and Forecasting

By analyzing long-term trends, Grafana helps organizations predict when they will run out of resources. This proactive approach prevents performance degradation before it affects users.

SLI/SLO Tracking

Grafana is often used to track Service Level Indicators (SLIs) and report on Service Level Objectives (SLOs).

- Error Budgets: Visualizing how much of an error budget remains for a given period.
- Availability Reports: Providing stakeholders with high-level summaries of system performance against business goals.

Logging Middleware — Centralized Logging

Need for Centralized Logging in Distributed Systems

In a monolithic application, all logs go to a single file on one server. Debugging is straightforward. But in a **distributed system with 50+ microservices**, each service runs on different containers, nodes, or even clouds. Logs are scattered everywhere.

Problems without centralized logging:

- An engineer must SSH into each server individually to read logs.
- When a request fails, it might span 5 services — correlating logs across them is nearly impossible manually.
- Container logs disappear when a pod restarts (in Kubernetes).
- No search capability across all logs simultaneously.
- No alerting on log content (e.g., "alert when 'out of memory' appears").

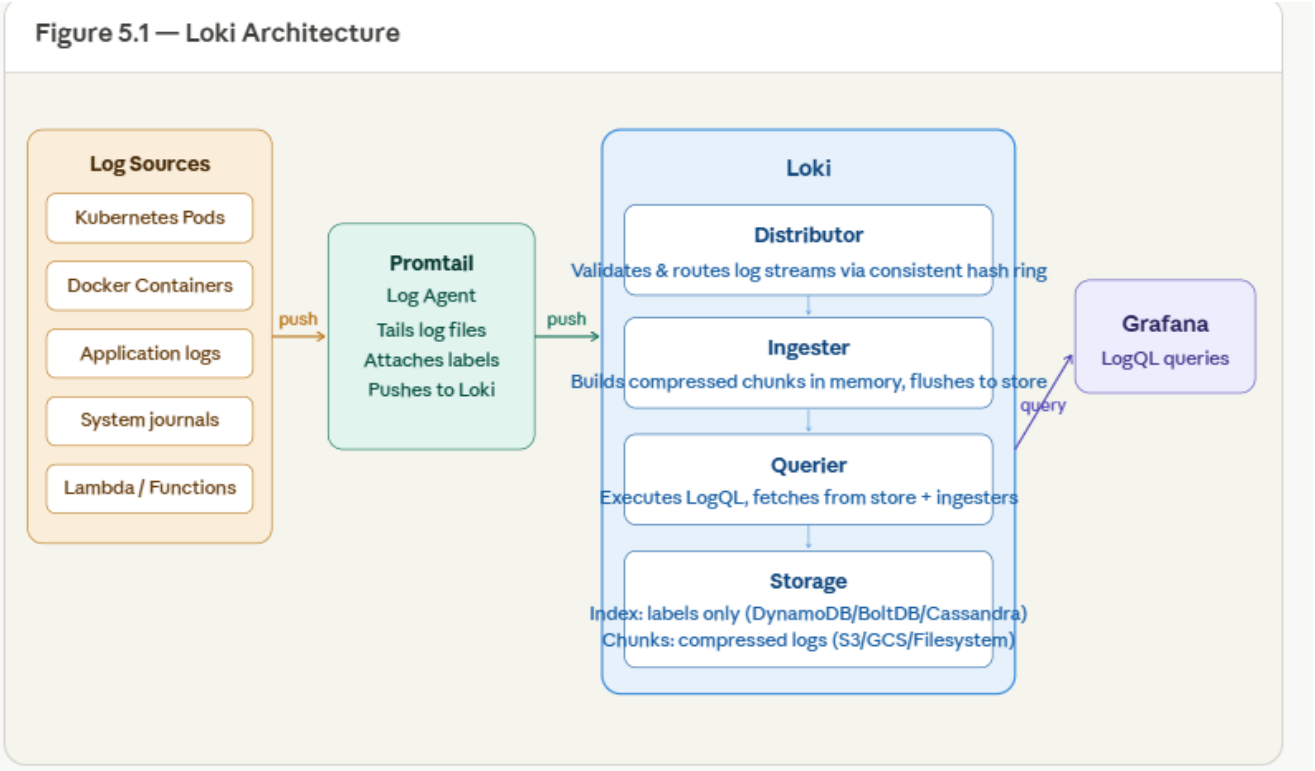
Centralized Logging solves this by:

Collecting logs from all services → Storing them in one place → Providing powerful search, filtering, and alerting → Correlating logs with metrics and traces for full observability.

Loki Architecture

Grafana Loki is a **horizontally scalable, highly available log aggregation system** inspired by Prometheus. Its key design principle: *store only metadata (labels) in the index; store log content in compressed chunks*. This makes it dramatically cheaper than Elasticsearch for log storage.

Figure 5.1 — Loki Architecture



Loki Components Explained

- **Promtail:** A log agent deployed alongside your applications. It tails log files, attaches labels (metadata), and pushes log streams to Loki. In Kubernetes, it typically runs as a DaemonSet on every node.
- **Distributor:** The first Loki component that receives log streams. It validates the data and routes streams to the appropriate ingesters using a consistent hash ring (similar to Cassandra's ring architecture).
- **Ingester:** Receives logs from distributors, builds compressed log chunks in memory, and periodically flushes them to long-term storage. Also handles in-memory queries.
- **Querier:** Handles LogQL queries by fetching from both the ingesters (recent data in memory) and the storage backend (older data).
- **Query Frontend:** (Optional) Caches and parallelizes large queries by splitting them into smaller sub-queries.

Log Streams and Labels

The fundamental unit in Loki is a **log stream** — a set of log lines that share the same set of labels. Labels are key-value pairs attached to each stream that identify *where the logs came from*.

How it Works:

- Loki groups logs into streams based on their labels.
- Each stream is stored as a series of compressed chunks.
- If you have two logs with even one different label value, they belong to two different streams.

Example: Imagine you have an application named web-server running in two environments: prod and dev.

1. {app="web-server", env="prod"} → Stream A
2. {app="web-server", env="dev"} → Stream B

Even if the log messages are identical, they are stored in separate streams because their labels differ.

Example — Log Stream Definition

{app="payment-service", environment="production", namespace="payments", pod="payment-7f9d-xk2j"}

All log lines from this pod share these labels and form a single log stream.

Each line in the stream:

2024-11-15T10:32:01Z INFO Payment processed: order_id=ORD-4892, amount=₹1499

2024-11-15T10:32:02Z ERROR Timeout connecting to bank API: retrying...

2024-11-15T10:32:05Z ERROR Bank API failed after 3 retries

Labels

Labels are key-value pairs attached to log lines. They are the only thing Loki indexes. They serve two primary purposes:

- Discovery: Finding which logs belong to which application or service.
- Sharding: Distributing logs across different storage nodes.

Static vs. Dynamic Labels

Type	Description	Examples
Static Labels	Fixed metadata usually set at the source (e.g., by Promtail or Alloy).	cluster, job, env, app
Dynamic Labels	Extracted from the log line itself during ingestion or at query time.	level, method, status_code

Detailed Examples of Labels

To understand labels, let's look at a real-world Kubernetes scenario.

Good Labels (Low Cardinality)

These have a limited number of unique values and are excellent for indexing.

- app="order-service": Identifies the application.
- env="production": Identifies the environment.
- namespace="backend": Identifies the K8s namespace.
- container="nginx": Identifies the specific container.
- region="us-east-1": Identifies the cloud region.

Bad Labels (High Cardinality)

These have a massive number of unique values. Avoid using these as labels!

- user_id="12345": Millions of users = millions of streams.
- request_id="abc-789": Every request is unique = infinite streams.
- client_ip="192.168.1.1": Thousands of unique IPs.

The Concept of Cardinality

Cardinality refers to the number of unique combinations of label names and values.

The Cardinality Formula:

If you have:

- app: 10 values
- env: 3 values
- cluster: 2 values

Total Cardinality = $10 \times 3 \times 2 = 60$ streams. This is healthy.

Cardinality Explosion:

If you add user_id(100,000 values) as a label:

Total Cardinality = $60 \times 100,000 = 6,000,000$ streams.

Why is this bad?

- **Memory Usage:** Loki keeps an index of every stream in memory. Too many streams will crash the system (OOM).
- **Query Performance:** Searching through millions of small streams is much slower than searching through a few large ones.
- **Storage Overhead:** Each stream creates its own chunks, leading to many small, inefficient files.

LogQL — Loki's Query Language

LogQL is Loki's query language, **inspired by PromQL**. It has two types of queries: Log queries (return raw log lines) and Metric queries (derive numbers from logs for graphing).

Basic Idea

LogQL works in **two stages**:

1. **Select log streams using labels**
2. **Filter log content using conditions**

```
{app="frontend"}
```

This means:

- Select frontend logs
- Show only logs containing **"error"**

1. Log Stream Selectors

The stream selector is **mandatory** in every LogQL query. It selects which log streams to query using **label matchers**.

Label Matchers

Operator	Meaning	Example
=	Exact match	app="nginx"
!=	Not equal	env!="dev"
=~	Regex match	app=~"web.*"
!~	Regex not match	host!~"test.*"

Examples

Select all logs from the nginx app

{app="nginx"}

Select logs from multiple apps using regex

{app=~"nginx|apache"}

Select production logs only

{app="api", env="production"}

Exclude staging logs

{app="backend", env!="staging"}

2 Log Pipeline Stages

After the stream selector, you chain **pipeline stages** using the | (pipe) operator.

Line Filter Expressions

The simplest filter — searches raw log lines for a string or pattern.

Operator	Meaning
|=	Line contains string
!=	Line does not contain string
|~	Line matches regex
!~	Line does not match regex

Logs containing "error"

{app="api"} |= "error"

Logs NOT containing "health-check"

```
{app="nginx"} != "health-check"

# Logs matching a regex pattern
{app="backend"} |~ "status=(4|5)[0-9]{2}"

# Chaining multiple filters
{app="api"} |= "error" != "timeout" |~ "user_id=[0-9]+"
```

3 Label Filter Expressions

After parsing, filter log lines based on **extracted label values**:

```
# Numeric comparison
{app="api"} | json | status_code >= 400

# String match
{app="api"} | logfmt | level="error"

# Duration comparison
{app="backend"} | logfmt | duration > 2s

# Multiple conditions
{app="api"} | json | status_code >= 500 | method="POST" | latency_ms > 1000
# Regex on label value
{app="api"} | json | path =~ "/api/v[12]/.*"
```

- **Example (Simple)**

Log Queries (Stream Selector + Filter)

Every LogQL query starts with a **stream selector** in curly braces — this selects which log streams to query. Then you can optionally chain filter expressions.

LogQL Syntax

```
{stream_selector} | filter_expression | filter_expression
```

Types of LogQL Queries

There are **two main types**:

1. **Log Queries (Log Filtering)**

Used to **view logs**

🔹 **Syntax**

{label_filters} [operator] "text"

Operators

Operator	Meaning
`	=`
!=	not equal
`	~`
!~	regex not match

Examples

✓ **Example 1**

{app="backend"} |= "error"

👉 Shows error logs from backend

Example 2

{env="prod"} != "debug"

👉 Excludes debug logs

✓ **Example 3**

{app="frontend"} |~ "timeout|failed"

👉 Matches logs containing:

- timeout OR failed

2️⃣ **Metric Queries (Log to Metrics)**

Used to **convert logs into metrics (numbers)**

🔹 **Example**

count_over_time({app="frontend"}[1m])

👉 Counts number of logs in 1 minute

🔹 **More Examples**

✓ **Error count**

count_over_time({app="frontend"} |= "error" [5m])

👉 Number of error logs in 5 minutes

✓ Rate of logs

rate({app="backend"}[1m])

👉 Logs per second

🔷 LogQL Query Structure

{labels} | filter | parser | aggregation

🔹 Example Breakdown

{app="frontend"} |= "error"

Part	Meaning
{app="frontend"}	Select stream
`	= "error"`

LogQL Filter Operations

Operator	Type	Example
|=	Contains (exact string)	{app="x"} |= "error"
!=	Does not contain	{app="x"} != "healthcheck"
|~	Matches regex	{app="x"} |~ "5[0-9]+"
!~	Does not match regex	{app="x"} !~ "GET /health"
| json	Parse JSON log line	extracts all JSON fields
| logfmt	Parse logfmt format	extracts key=value pairs
| pattern	Parse by pattern	| pattern '<ip> - <user>'
| line_format	Reformat log line	Go template string
| label_format	Rename/drop labels	| label_format host=ip

LogQL Examples

```
# 1. Select all logs from a specific app
{app="order-service"}

# 2. Filter logs containing a keyword
{app="order-service"} |= "ERROR"

# 3. Exclude lines containing a word
{app="order-service"} != "DEBUG"

# 4. Regex filter — lines matching pattern
{app="order-service"} |~ "timeout|connection refused"

# 5. Case-insensitive regex
{app="order-service"} |~ "(?i)payment failed"

# 6. Parse JSON structured logs and filter by parsed field
{app="payment-svc"} | json | status_code >= 500

# 7. Parse logfmt format
{app="nginx"} | logfmt | method="POST" | status=500

# 8. Count log lines per minute (Metric Query)
rate({app="payment-svc"} |= "ERROR"[1m])

# 9. Count errors across all production services
sum(rate({environment="production"} |= "ERROR"[5m])) by (app)

# 10. Extract a field and aggregate
{app="api"} | json | line_format "{{.user_id}}: {{.response_time}}"

# 11. 99th percentile of extracted response time field
quantile_over_time(0.99, {app="api"} | json | unwrap response_time [5m])
```

Integration of Loki with Grafana

Step 1: Add Loki as a Data Source

1. Log in to Grafana and navigate to Connections > Data Sources.

2. Click Add data source and select Loki.
3. Enter the URL where Loki is reachable (e.g., http://loki:3110).
4. If Loki is configured with multi-tenancy, add the X-Scope-OrgID header.
5. Click Save & test. A green notification should appear if the connection is successful.

Step 2: Configure Derived Fields (Optional but Recommended)

Derived fields allow you to turn parts of a log line into clickable links. This is essential for linking logs to traces (Tempo) or external systems (GitHub, Jira).

- **Name:** The display name for the link (e.g., TraceID).
- **Regex:** A regular expression to extract the ID from the log (e.g., traceID=(\w+)).
- **URL:** The target link, using ${__value.raw} to insert the extracted ID.

Key Integration Features

The Explore View

The Explore mode in Grafana is the primary interface for interacting with Loki. It provides:

- **Log Browser:** A UI to select labels and discover available log streams without writing LogQL.
- **Live Tail:** Real-time streaming of logs directly into the browser.
- **Split View:** The ability to view metrics (Prometheus) on one side and logs (Loki) on the other, with synchronized time ranges.

Seamless Correlation (Metrics to Logs)

One of the most powerful features of the integration is the ability to jump from a metric spike to the corresponding logs.

1. In a Prometheus dashboard, click on a data point.
2. Select Explore or a pre-configured Data Link.
3. Grafana automatically carries over the labels and time range to a Loki query, showing you exactly what happened during that spike.

Log-Based Dashboards

Loki data can be used to create standard Grafana dashboard panels:

- **Logs Panel:** Displays the raw log lines with highlighting and filtering.
- **Time Series Panel:** Uses LogQL metric queries (e.g., rate({job="api"}[5m])) to visualize log volume or error rates over time.
- **Stat Panel:** Shows the total count of specific events (e.g., "Total 500 Errors in last 24h").

Comparison of Loki-based logging with Traditional ELK Stack

Aspect	Grafana Loki	ELK Stack
Indexing strategy	Labels only (metadata indexed; log content compressed)	Full-text indexing of all log content
Storage cost	Very low (10-20x cheaper than ELK)	High (Elasticsearch is storage-intensive)
Query language	LogQL (PromQL-inspired)	KQL / Lucene query syntax
Search capability	Regex/text search at query time (slower on large datasets)	Instant full-text search (indexes upfront)
Scalability	Horizontally scalable; microservices mode	Scalable but complex cluster management
Operational complexity	Low — single binary or microservices mode	High — manage ES cluster, shards, replicas, Logstash, Kibana separately
Prometheus integration	Native — same labels, same Grafana UI	Requires separate APM/Beats setup
Log agents	Promtail, Fluentd, Fluentbit, Vector	Filebeat, Logstash, Fluentd
Schema	Schema-less (labels + raw text)	Schema-based (mappings define fields)
Best for	Kubernetes, cloud-native, cost-sensitive, Prometheus-first orgs	Complex full-text log search, compliance, security (SIEM), rich analytics
License	Apache 2.0 / Grafana Enterprise	Elastic License 2.0 (partially open-source)